Tarea 1

Sistema de notificación de sismos
INF326
Arquitectura de Software

Yohanns Jara
201830035-7

Octubre de 2026

1. Arquitectura

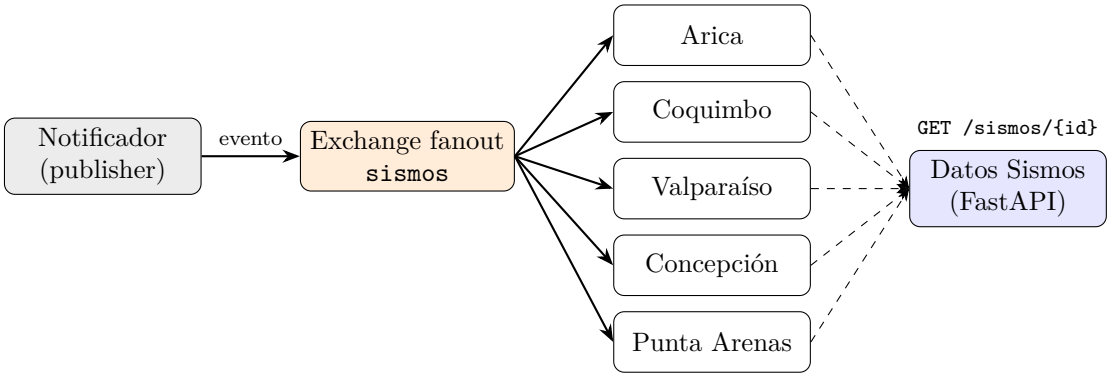


Figura 1: Componentes. Flechas continuas: mensajería (push del evento). Flechas punteadas: consulta HTTP (pull del detalle), solo si el sismo es de interés.

Flujo de un evento:

1. El notificador publica `{id, lat, lon, fecha_utc}` en el exchange `sismos`.
2. El exchange `fanout` copia el evento a la cola de cada suscriptor (cada ciudad tiene su propia cola).
3. Cada suscriptor calcula la distancia entre su ciudad y el epicentro (sección 3).
4. Si la distancia es ≥ 500 km, registra el evento como *ignorado* y termina, sin tráfico HTTP.
5. Si es < 500 km, registra que el evento es *de interés*, consulta `GET /sismos/{id}` y registra el detalle recibido.

2. Diseño de mensajes

2.1. Evento publicado

```
{"id": 1, "lat": -20.35, "lon": -70.45, "fecha_utc": "2026-10-01 03:12:41"}
```

Campo	Tipo	Por qué está
`id`	int	Identifica el sismo (incremental: 1, 2, 3...) y es la clave para pedir el detalle. Se omiten sistemas más complejos de hasheo o string compuesto por simplicidad en la tarea.
`lat`, `lon`	float	La latitud y longitud geografica del evento, el suscriptor las necesita para calcular la distancia y decidir si le interesa.
`fecha_utc`	string	Momento de ocurrencia del evento (no del envío). Permite ordenar eventos y descartar avisos atrasados. En UTC para no depender del huso horario.

Son los tres datos mínimos de un evento: **qué** ocurrió (id), **dónde** (lat/lon) y **cuándo** (fecha). Se excluyen a propósito la magnitud, profundidad, referencia y agencia: ningún suscriptor las necesita para *decidir* si le interesa el sismo, y quien se interesa las obtiene por *pull*. Así el mensaje no lleva información extra que no se use.

2.2. Detalle entregado por Datos Sismos

`GET /sismos/{id}` responde `200` con el registro completo (`404` si no existe), con campos al estilo de https://www.sismologia.cl:

```
{"id": 1, "fecha_local": "2026-10-01 00:12:41", "fecha_utc": "2026-10-01 03:12:41",
 "latitud": -20.35, "longitud": -70.45, "profundidad_km": 38.2, "magnitud": 5.8,
 "tipo_magnitud": "Mw", "referencia": "35 km al O de Iquique", "agencia": "CSN"}
```

`GET /sismos` entrega la lista completa.

2.3. Notificación de evento vs. transferencia de estado

- **Event notification (elegido, modelo pull):** el evento solo avisa que algo ocurrió y trae lo necesario para filtrar; el detalle se pide bajo demanda. Mensajes pequeños, una fuente de verdad para el detalle y datos corregibles.
- **Event-carried state transfer (modelo push):** el evento lleva el registro completo. Evita la consulta HTTP, pero envía datos a quienes los descartan, duplica el estado en cada consumidor y puede quedar obsoleto si el dato se corrige.

Cada sismo interesa solo a las ciudades a menos de 500 km. El modelo push conviene cuando casi todos los consumidores necesitan el detalle; el pull, cuando pocos lo necesitan, que es este caso.

3. Cálculo de la distancia

El enunciado pide una distancia geodésica. Se usa la fórmula de **Haversine**, que entrega la distancia sobre la superficie de una esfera (el arco de círculo máximo entre dos puntos). Con φ la latitud y λ la longitud, ambas en radianes:

$$a = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos\varphi_1 \, \cos\varphi_2 \, \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right) \tag{1}$$

$$d = 2R \arcsin\left(\sqrt{a}\right), \qquad R = 6371 \text{ km} \tag{2}$$

- a es el cuadrado de la mitad de la cuerda entre los dos puntos, en una esfera de radio 1.
- $2\arcsin(\sqrt{a})$ es el ángulo central entre los puntos, y multiplicado por R da la longitud del arco.

La Tierra no es una esfera perfecta. Frente a un cálculo sobre el elipsoide WGS-84 (Vincenty) el error es del orden de 0,5 %, es decir, unos pocos km. Es irrelevante para un umbral de interés de 500 km y evita una dependencia externa.

4. Implementación

Archivo	Responsabilidad
`notificador.py`	Menú interactivo que publica eventos en el exchange `sismos`.
`subscriber.py`	Suscriptor por ciudad: filtra por distancia y hace el pull del detalle.
`datos_sismos.py`	Servicio HTTP FastAPI sobre `sismos.json`.
`logs.py`	Formato de logs con color, compartido por notificador y suscriptores.
`sismos.json`	Catálogo de sismos (un registro por línea).
`config/`	`config_reader.py` arma la URL de conexión a CloudAMQP desde `config.ini` (credenciales fuera del código); `config.ini.example` es la plantilla.

4.1. Notificador

Ciclo `while True` con un menú:

1. **Enviar todos secuencialmente**, con una pausa de 1 s entre eventos para que los logs se lean en orden.
2. **Enviar uno en particular**, elegido de una lista numerada.
3. **Crear un sismo personalizado**: pide latitud, longitud, profundidad, magnitud y referencia; asigna el siguiente id y la fecha actual, lo agrega al final de `sismos.json` y lo publica.

Abre una conexión por envío: mientras el menú espera la entrada del usuario, `BlockingConnection` no responde los heartbeats y el broker cerraría una conexión que quedara abierta.

4.2. Suscriptores

`python Tarea1/subscriber.py <Ciudad>` levanta una ciudad. Sin argumento levanta las cinco en un solo proceso, con una conexión pero **una cola exclusiva por ciudad**, de modo que cada una recibe su propia copia del evento. Con una cola compartida serían *competing consumers* y cada evento llegaría a una sola ciudad.

4.3. Logs

Cada línea tiene timestamp y el nombre del componente. En consola, el timestamp va en gris, cada ciudad con un color propio, `ignorado` en rojo y `de interés` en verde. Ejemplo para el sismo 1 (cerca de Iquique):

```
2026-10-08 17:58:38,264 [arica] sismo 1 (2026-10-01 03:12:41 UTC) de interés (a 209 km): fetch GET /sismos/1
2026-10-08 17:58:38,461 [arica] sismo 1 detalle recibido: {'id': 1, ..., 'magnitud': 5.8, ...}
2026-10-08 17:58:38,463 [coquimbo] sismo 1 (2026-10-01 03:12:41 UTC) ignorado (a 1072 km)
2026-10-08 17:58:38,464 [valparaiso] sismo 1 (2026-10-01 03:12:41 UTC) ignorado (a 1415 km)
2026-10-08 17:58:38,465 [concepcion] sismo 1 (2026-10-01 03:12:41 UTC) ignorado (a 1849 km)
2026-10-08 17:58:38,466 [punta arenas] sismo 1 (2026-10-01 03:12:41 UTC) ignorado (a 3649 km)
```

4.4. Datos Sismos

Relee `sismos.json` en cada consulta, para ver los sismos personalizados que agrega el notificador. El notificador escribe primero en un archivo temporal y luego lo reemplaza de forma atómica, para que el servicio nunca lea un archivo a medio escribir. No hay `POST`: el enunciado indica que los datos ya están en el servicio.

5. Instrucciones de ejecución

5.1. Prerrequisitos

- Python 3.10 o superior con `pika`, `fastapi` y `uvicorn` (`pip install -r Tarea1/requirements.txt`).
- Una instancia de RabbitMQ (CloudAMQP) y el archivo `Tarea1/config/config.ini`.

```
[CONNECTION]
USER_PASSWORD = usuario:clave
SERVER = host.rmq.cloudamqp.com
PORT = 5671
VHOST = vhost
```

- Ejecutar los comandos desde la raíz del repositorio. La tarea es autocontenida: los scripts importan `config` desde su propia carpeta.

5.2. Comandos

Terminal	Comando
1	`uvicorn datos_sismos:app -port 8000`
2	`python subscriber.py` (las 5 ciudades) o `python subscriber.py <Ciudad>`
3	`python notificador.py` (Ejecutar alguna de las opciones)

Orden: 1, 2 y al final 3, porque las colas de los suscriptores deben existir antes de publicar. Si el servicio no corre en `localhost:8000`, se define la variable `API_URL` (por ejemplo `http://127.0.0.1:8001`).

6. Consideraciones de implementación

- **Fanout en vez de topic:** cada suscriptor decide por sí mismo; el publicador no conoce la geografía de los suscriptores.
- **Colas exclusivas sin nombre:** se eliminan al desconectarse el suscriptor, por lo que un suscriptor caído no recibe los eventos publicados mientras estaba fuera.
- **`auto_ack`:** el mensaje se confirma al recibirlo. Es simple, pero da entrega *at-most-once*: si el procesamiento falla, el evento se pierde.
- **Errores del pull:** si el detalle responde `404`, hay timeout (5 s) o el servicio no responde, el suscriptor lo informa y sigue escuchando; no hay reintentos.
- **Sin autenticación** en el servicio HTTP

7. Discusión de trade-offs

7.1. Desacoplamiento pub-sub vs. garantías de entrega

Se gana: el notificador no conoce a los suscriptores; se agregan o quitan ciudades sin tocarlo. **Se pierde:** con colas exclusivas no durables y `auto_ack` la entrega es *at-most-once*: si un suscriptor está caído o falla al procesar, el evento se pierde sin rastro. **¿Importa?** Sí: en alertas, perder un aviso es grave. Aquí se acepta por ser una demostración. La mitigación es usar colas durables con nombre fijo, mensajes persistentes (`delivery_mode=2`), exchange durable y ack manual después de procesar (*at-least-once*), con dead-letter queue. El costo es que las colas se acumulan si un suscriptor no vuelve, y que pueden llegar duplicados.

7.2. Filtrado en el suscriptor vs. en el broker

Se gana: publicador y broker simples; el criterio de interés vive en cada consumidor y cambia sin tocar el resto. **Se pierde:** cada evento llega a los 5 suscriptores aunque a la mayoría no le sirva. Un exchange `topic` con claves por zona filtraría en el broker, pero obliga al publicador a conocer la grilla geográfica, y un radio de 500 km cruza varias zonas (cada suscriptor necesitaría varios bindings). **¿Importa?** Poco: los sismos son esporádicos, los mensajes pesan decenas de bytes y el filtro es un cálculo barato.

7.3. Evento delgado + pull vs. evento completo

Se gana: mensajes pequeños, datos corregibles (el CSN revisa magnitudes tras el primer reporte y con pull se obtiene la versión vigente) y solo los interesados pagan el costo del detalle. **Se pierde:** una consulta HTTP extra (latencia) y un nuevo punto de falla. Tras un sismo grande todos los interesados consultan a la vez (*thundering herd*) y el servicio es un SPOF: si cae, los suscriptores conocen el evento pero no su detalle. Además, la ubicación del evento también puede corregirse, así que el filtro usa un dato que puede quedar desactualizado respecto del detalle. **¿Importa?** La carga no, con pocos suscriptores; la disponibilidad del servicio sí en producción (caché, réplicas, reintentos con backoff).

7.4. Latencia

El pull agrega una consulta HTTP. En alerta temprana o de tsunami cada segundo cuenta y sería inaceptable. Este sistema entrega información de un sismo que ya ocurrió y se registró, por lo que unos milisegundos más no importan.

7.5. Consistencia entre evento y dato

El evento puede llegar antes de que el dato exista en Datos Sismos (`404`) o el dato puede cambiar tras la notificación (consistencia eventual). Se resuelve con reintentos con espera o publicando solo después de persistir el dato, que es lo que hace el notificador con los sismos personalizados: primero los guarda y luego publica.

7.6. Otros

- **Seguridad:** sin autenticación ni TLS en el servicio HTTP; en producción se necesitarían, junto con límites de tasa.
- **Datos en archivo JSON:** suficientes para la demostración, pero sin escrituras concurrentes ni escalamiento horizontal.
- **Duplicados y orden:** con ack manual pueden llegar duplicados; el `id` permite deduplicar (idempotencia) y `fecha_utc` permite ordenar.

8. Back of the Envelope

Una estimación de orden de magnitud permite decidir si basta una arquitectura simple. Se plantea cómo estimar el uso con información del fenómeno, sin calcular valores.

8.1. Tasa de eventos

- **Promedio:** contar en el catálogo del CSN los sismos en una ventana T: $\lambda_{prom} = N_{eventos}/T$.
- **Umbral de magnitud:** la ley de Gutenberg-Richter, $\log_{10} N(\geq M) = a - bM$ (con $b \approx 1$), da la tasa sobre un umbral M_{min}: $\lambda(M_{min}) = 10^{a-bM_{min}}$, ajustando a y b al catálogo.
- **Peak:** tras un terremoto grande, las réplicas siguen la ley de Omori, $n(t) = K/(c+t)^p$ con $p \approx 1$. La tasa peak es $\lambda_{peak} \approx K/c$. Se dimensiona con el peak, no con el promedio.

8.2. Tamaño de mensaje y ancho de banda

$$B_{pub} = \lambda \cdot S_{evento} \qquad\qquad B_{fanout} = \lambda \cdot S_{evento} \cdot N_{subs}$$

donde S_{evento} son los bytes del JSON `{id, lat, lon, fecha_utc}` más la sobrecarga de AMQP. Para comparar con el modelo push se reemplaza S_{evento} por $S_{detalle}$.

8.3. Solicitudes HTTP

Sea $p_i = P(d < 500 \text{ km})$ la fracción de sismos que interesa al suscriptor i. Se estima contando en el catálogo los eventos a menos de 500 km de cada ciudad, o con el área del círculo de 500 km ponderada por la distribución espacial de la sismicidad (concentrada en la zona de subducción). Entonces:

$$\lambda_{HTTP} = \lambda \sum_{i=1}^{N_{subs}} p_i = \lambda \cdot N_{subs} \cdot \bar{p}, \qquad\qquad B_{HTTP} = \lambda_{HTTP} \cdot S_{detalle}$$

Tras un sismo grande llegan casi a la vez unas $N_{subs} \cdot \bar{p}'$ consultas, con $\bar{p}' > \bar{p}$ porque un sismo grande interesa a más distancia: ese es el caso de ráfaga.

8.4. Escalamiento

Con N suscriptores, λ_{HTTP} y B_{fanout} crecen linealmente en N. Con la capacidad de una instancia HTTP C_{HTTP} (req/s, medida con una prueba de carga) y del broker C_{broker} (msg/s), se verifica:

$$\lambda_{HTTP}^{peak} \le C_{HTTP}, \qquad \lambda^{peak} \cdot N \le C_{broker}, \qquad \text{instancias} = \left\lceil \lambda_{HTTP}^{peak} / C_{HTTP} \right\rceil$$

8.5. Conclusión

Con sismos notificables esporádicos (incluso en ráfagas de réplicas) y $N_{subs} = 5$, se espera que un broker y una instancia FastAPI queden muy por debajo de su capacidad, lo que justifica la arquitectura simple. Dejaría de bastar con millones de suscriptores: el fanout y el pull simultáneo tras un sismo grande saturarían el servicio. Ahí se necesitaría caché o CDN para el detalle (idéntico para todos), réplicas con balanceo, filtrado en el broker o eventos completos que eviten el pull.